

Phase 4 - Room Decor & Shop (Living Room)

MVP: generate ONLY a simple flat brick wall in PixelLab, draw the floor in code (SimpleFloor, swappable later), and make furniture purchasable + draggable decor. You can build the WHOLE gameplay now with placeholder decor boxes and swap real art in later. Mixed currency: cheap = scrap, premium = premium gems. Placement is stored locally in Isar; ownership and spending go through the Firestore economy.

Decisions

- *Placement: free drag-and-drop (Decorate mode)*
- *Scope: Living Room only (MVP)*
- *Currency: mixed - scrap (cheap) + premium gems (premium)*
- *Background: wall (PixelLab) + floor in code SimpleFloor (swap color/tile later, one block)*
- *Game first: build decor with placeholder boxes, plug real art in later*

1) Art direction + sizes

- *Wall: a simple flat brick wall only. View = Side (Sidescroller), Direction = None, Remove bg OFF.*
- *Wall in-game 400x350 (50/50 layout). Ideal canvas ~200x175 (ratio 8:7). 180x216 also works - minor brick stretch is barely visible.*
- *Floor: NOT generated; drawn in code via SimpleFloor. Changing color/tile later is one block.*
- *Decor sprites: one PNG per item, transparent, Remove bg ON. No need to make them yet - start with placeholder boxes (section 4), add art later.*

Wall prompt (single line):

Flat 2D side-scroller game wall texture, a simple plain brick wall, orthographic front view, fills the entire image as a wall only, NO floor, NO ceiling, NO side walls, NO perspective, plain uniform brick pattern, minimal detail, dark moody tone with a faint subtle cyan neon edge light, flat even lighting, pixel art, 16-bit, no furniture, no characters, no animals, no text.

Decor sprite prompt template (later, Remove bg ON):

A single <ITEM> object only, centered, isolated on a plain transparent background, no room, no floor, no background shadow, cyberpunk neon style with cyan and magenta accents, pixel art, 16-bit, front-facing game asset.

Replace <ITEM> with: neon couch, holographic TV, potted plant, floor lamp, round rug, neon wall sign.

2) Data model

```
enum DecorCategory { seating, screen, plant, lighting, storage, rug, wallArt }

class DecorDef {
    final String id;
    final String name;
    final String asset;           // example: decor/couch.png
    final DecorCategory category;
    final CurrencyType currency;  // scrap or premiumGems
    final int price;
    final Vector2 defaultSize;
    final Vector2 defaultPos;
    final int layer;              // render priority inside the room
    const DecorDef({
        required this.id,
        required this.name,
        required this.asset,
        required this.category,
        required this.currency,
        required this.price,
        required this.defaultSize,
        required this.defaultPos,
        this.layer = 0,
    });
}
```

Placement (Isar, local cosmetic):

```
@collection
class PlacedDecor {
    Id isarId = Isar.autoIncrement;
    @Index() late String itemId;    // matches DecorDef.id
    late String room;              // for now: living
    late double x;
    late double y;
    double scale = 1.0;
    bool flipped = false;
    int layer = 0;
}
```

Why split: ownership + spending live in the economy (Firestore, economyVersion bumps, authoritative). Placement is purely visual so it stays local in Isar (offline-first, fast).

3) Living Room decor catalog (sample)

Item	Currency	Price	Spot
Neon Couch	<i>scrap</i>	<i>120</i>	<i>floor</i>
Holo TV	<i>premium gems</i>	<i>15</i>	<i>wall</i>
Potted Plant	<i>scrap</i>	<i>60</i>	<i>floor</i>
Floor Lamp	<i>scrap</i>	<i>80</i>	<i>floor</i>
Round Rug	<i>scrap</i>	<i>40</i>	<i>floor</i>
Neon Sign	<i>premium gems</i>	<i>10</i>	<i>wall</i>

```

const kLivingRoomDecor = <DecorDef>[
  DecorDef(id: 'couch', name: 'Neon Couch', asset: 'decor/couch.png',
    category: DecorCategory.seating, currency: CurrencyType.scrap, price: 120,
    defaultSize: Vector2(150, 70), defaultPos: Vector2(0, 300), layer: 0),
  DecorDef(id: 'holo_tv', name: 'Holo TV', asset: 'decor/holo_tv.png',
    category: DecorCategory.screen, currency: CurrencyType.premiumGems, price: 15,
    defaultSize: Vector2(140, 90), defaultPos: Vector2(0, -120), layer: -2),
  DecorDef(id: 'plant', name: 'Potted Plant', asset: 'decor/plant.png',
    category: DecorCategory.plant, currency: CurrencyType.scrap, price: 60,
    defaultSize: Vector2(50, 80), defaultPos: Vector2(140, 320), layer: 1),
  DecorDef(id: 'lamp', name: 'Floor Lamp', asset: 'decor/lamp.png',
    category: DecorCategory.lighting, currency: CurrencyType.scrap, price: 80,
    defaultSize: Vector2(40, 110), defaultPos: Vector2(-150, 320), layer: 1),
  DecorDef(id: 'rug', name: 'Round Rug', asset: 'decor/rug.png',
    category: DecorCategory.rug, currency: CurrencyType.scrap, price: 40,
    defaultSize: Vector2(220, 70), defaultPos: Vector2(0, 340), layer: -5),
  DecorDef(id: 'neon_sign', name: 'Neon Sign', asset: 'decor/neon_sign.png',
    category: DecorCategory.wallArt, currency: CurrencyType.premiumGems, price: 10,
    defaultSize: Vector2(90, 40), defaultPos: Vector2(120, -180), layer: -3),
];

```

4) DecorComponent (placeholder-ready)

```

class DecorComponent extends SpriteComponent
  with DragCallbacks, HasGameReference<RoboPetGame> {
  DecorComponent({required this.def, required Vector2 position, this.onMoved})
    : super(position: position, anchor: Anchor.bottomCenter, priority: def.layer);

  final DecorDef def;
  final void Function(DecorComponent)? onMoved;
  bool editable = false;

  @override
  Future<void> onLoad() async {
    size = def.defaultSize;
    try {
      sprite = await Sprite.load(def.asset);
    } catch (_) {
      // PLACEHOLDER: colored box + name until art exists.
      // When the PNG is added, the real sprite loads automatically; no code change.
      add(RectangleComponent(
        size: size,
        paint: Paint()..color = const Color(0xFF394067),
      ));
      add(TextComponent(text: def.name, anchor: Anchor.center, position: size / 2));
    }
  }

  // Only capture drags/taps in Decorate mode
  @override
  bool containsLocalPoint(Vector2 point) =>
    editable && super.containsLocalPoint(point);

  @override
  void onDragUpdate(DragUpdateEvent event) {
    if (!editable) return;
    position += event.localDelta;
    position.x = position.x.clamp(-180.0, 180.0); // room width
    position.y = position.y.clamp(-300.0, 345.0); // wall top to floor bottom
  }
}

```

```

@override
void onDragEnd(DragEndEvent event) {
    if (editable) onMoved?.call(this);
}
}

```

Game first: thanks to try/catch, missing PNGs render as a colored box + name. Build the entire buy/place/drag loop now; dropping decor PNGs into assets/images/decor/ later swaps in the real sprite with zero code change.

5) LivingRoomWorld wiring (wall + SimpleFloor, 50/50)

```

// Simple, non-distracting floor. Change its color later, or swap to
// SpriteComponent('rooms/living_floor.png') for a tiled texture.
class SimpleFloor extends PositionComponent {
    SimpleFloor({required Vector2 size, required Vector2 position})
        : super(size: size, position: position, priority: -9);

    final _floor = Paint()..color = const Color(0xFF161A2E);
    final _shade = Paint()..color = const Color(0x22000000);
    final _edge = Paint()
        ..color = const Color(0xAA22E1FF)
        ..strokeWidth = 2;

    @override
    void render(Canvas canvas) {
        final w = size.x, h = size.y;
        canvas.drawRect(Rect.fromLTWH(0, 0, w, h), _floor);
        canvas.drawRect(Rect.fromLTWH(0, h * 0.5, w, h * 0.5), _shade);
        canvas.drawLine(const Offset(0, 0), Offset(w, 0), _edge);
    }
}

class LivingRoomWorld extends World with HasGameReference<RoboPetGame> {
    final List<DecorComponent> _decor = [];
    late final RobotDog _dog;

    @override
    Future<void> onLoad() async {
        // WALL: upper half (400x350)
        add(SpriteComponent(
            sprite: await Sprite.load('rooms/living_wall.png'),
            size: Vector2(400, 350),
            position: Vector2(-200, -350),
            priority: -10,
        ));

        // FLOOR: lower half, drawn in code (50/50)
        add(SimpleFloor(size: Vector2(400, 350), position: Vector2(-200, 0)));

        // saved decor
        final repo = game.ref.read(decorRepositoryProvider);
        for (final p in await repo.loadPlaced('living')) {
            final def = kLivingRoomDecor.firstWhere((d) => d.id == p.itemId);
            final c = DecorComponent(
                def: def,
                position: Vector2(p.x, p.y),
                onMoved: (comp) => repo.savePlacement('living', comp),
            );
            if (p.flipped) c.flipHorizontallyAroundCenter();
        }
    }
}

```

```

    _decor.add(c);
    add(c);
  }

  _dog = RobotDog(position: Vector2(0, 280)); // on the floor
  add(_dog);
}

void addDecor(DecorComponent c) {
  _decor.add(c);
  add(c);
  game.ref.read(decorRepositoryProvider).savePlacement('living', c);
}

void setEditMode(bool on) {
  for (final c in _decor) c.editable = on;
  if (on) { _dog.stop(); } else { _dog.startWander(); }
}
}

```

Floor tuning / swapping: for the split, change wall size.y (350) and floor position.y together (-350 + wallY = floorY). Color: the _floor Paint. Tile: replace the SimpleFloor line with SpriteComponent(rooms/living_floor.png, size 400x350, pos (-200,0)) - nothing else changes. Dog y 260-300 sits on it.

6) Shop + purchase (economy-backed)

```

Future<bool> buyDecor(WidgetRef ref, DecorDef def) async {
  final economy = ref.read(economyServiceProvider);
  final ok = await economy.spend(
    currency: def.currency, // scrap or premiumGems
    amount: def.price,
    reason: 'decor:${def.id}',
  );
  // bumps economyVersion, writes Firestore
  if (!ok) return false; // not enough currency
  await economy.grantDecor(def.id); // ownership in Firestore
  return true;
}

```

- *Shop UI overlay: grid of kLivingRoomDecor; each card shows price + currency icon. Buy disabled if balance is too low.*
- *Owned items show a Place button -> world.addDecor(DecorComponent(def: def, position: def.defaultPos.clone(), ...)).*
- *Decorate button: calls world.setEditMode(true/false); in edit mode the dog stops and decor becomes draggable.*

7) Cursor task (paste)

1. (Art OPTIONAL, can skip for now) PixelLab: wall for 400x350 (canvas ~200x175, View Side, Direction None, Remove bg OFF) -> assets/images/rooms/living_wall.png. Decor PNGs later (Remove bg ON) -> assets/images/decor/. With no art, the placeholder box kicks in automatically.
2. Register assets/images/rooms/ and assets/images/decor/ in pubspec assets, run flutter pub get.
3. Add DecorCategory enum, DecorDef class, kLivingRoomDecor catalog.

```
4. Add Isar PlacedDecor collection + DecorRepository (loadPlaced, savePlacement,
removePlacement). Run build_runner for Isar codegen.
5. Add DecorComponent (SpriteComponent + DragCallbacks, editable flag, placeholder via
try/catch, clamp x -180..180 and y -300..345, onMoved persists).
6. Add SimpleFloor + update LivingRoomWorld: wall SpriteComponent 400x350 at (-200,-350)
priority -10, SimpleFloor 400x350 at (-200,0), load placed decor, add RobotDog at y 280,
add addDecor and setEditMode.
7. Add a Decorate toggle button overlay calling world.setEditMode; dog stops while
editing.
8. Add a Decor Shop overlay: grid of kLivingRoomDecor with price and currency icon; Buy
calls economy.spend then grantDecor; owned items show Place which calls world.addDecor at
defaultPos.
9. Hot-restart and test with placeholder boxes: buy, place, drag, exit decorate mode;
confirm dog wanders on the floor and placement persists after restart.
```

8) Build order (game first, art later)

- *Wall (400x350) + SimpleFloor (400x350 at y=0) + dog y=280 -> already done, 50/50.*
- *DecorDef + catalog + Isar PlacedDecor + repo (build_runner).*
- *DecorComponent (placeholder box) + Decorate mode + Shop overlay.*
- *Test with placeholders: buy -> place -> drag -> exit -> confirm persistence after restart.*
- *Later: generate real decor PNGs into decor/ -> real sprites appear automatically, no code change.*